**Национальный исследовательский ядерный университет «МИФИ»**
Институт физико-технических интеллектуальных систем (ИФТИС)
Кафедра **«Автоматика»**

# ОТЧЕТ ПО ЗАДАНИЮ НА УЧЕБНУЮ ПРАКТИКУ

по направлению подготовки: Системный анализ и управление. Информационно-измерительные и управляющие системы киберфизических устройств и установок.

Выполнил(а): Шелманова Ева Олеговна
Группа: Б23-605

Руководитель практики: Толоконский Андрей Олегович, к.т.н., доцент кафедры “Теплофизика"

г. Москва
2026

Задание на учебную практику (ознакомительную) (6 семестр)

| | | | |
|---|---|---|---|
| Студент: | Шелманова Ева Олеговна | Группа | Б23-605 |
| Тема: | Разработка интерактивного ассистента для операторов АЭС | | |

1. Исходные данные

1.1. Объект исследования

Операционный информационный поток и нормативно-техническая документация АЭС: операционные процедуры, аварийные инструкции, регламенты, справочные таблицы.

1.2. Предмет исследования (разработки)

Прототип распределённой RAG-системы в агентном оформлении для: быстрого поиска релевантных фрагментов регламентов; генерации ответов с явными ссылками на источники; автоматической верификации ответов по правилам и метрикам достоверности.

1.3. Основные технические требования, предъявляемые к исследованию (разработке)

Достоверность: ссылки на источники в каждом ответе. Трассируемость: лог всех операций. Надёжность retrieval: precision@1 ≥ порога, быстрый отклик. Модульность: разделение агентов на роли. Безопасность UI: экран «источники и ограничения», пометки low-confidence/conflict. Реплицируемость: запуск по инструкциям.

2. Содержание работы

2.1. Теоретическая часть (обзор источников по выбранной тематике; описание объекта; выбор структуры и элементной базы устройства или системы;

проведение расчетов; разработка математических моделей, схем, алгоритмов, методик исследования и проектирования и т.д.)

– Обзор литературы: RAG, dense/sparse retrieval, n8n, верифицируемость LLM, агентные архитектуры, регламентные базы знаний в критичных отраслях.

– Описание объекта: информационные потоки оператора АЭС, типы документов (процедуры, регламенты, техспецификации) и запросов (информационные, процедурные, troubleshooting, emergency).

– Архитектура: парсер → chunker → embeddings → vector DB → retriever → generator → verifier → UI/API; обоснование выбора инструментов.

– Модели и алгоритмы: гибридное индексирование (dense + BM25), reranking, prompt-engineering для citation-aware генерации, верификация (шаблоны, проверки диапазонов, детектор конфликтов).

– Метрики: precision@k, recall@k, ROUGE/F1, hallucination rate, time-to-answer, usability. Безопасность: ограничения LLM, риски в критичных сценариях, human-in-the-loop.

2.2. Практическая часть (моделирование, макетирование, верификация, наладка, проведение различного рода испытаний и т.д.)

– Корпус документов: парсинг PDF/HTML, очистка, chunking (500–1000 токенов), метаданные.

– Индекс и retrieval: embeddings + FAISS, BM25 fallback.

– Генератор ответов: LLM с prompt-шаблонами, обязательные ссылки на источники.

– Verifier-agent: unit checks, числовые диапазоны, детектор конфликтов, fact-checker.

– Оркестрация агентов: логика вызовов агентов, при конфликте – оба фрагмента с пометкой.

– Интерфейс: веб-приложение с поиском, ответом, источниками и индикатором верификации.

– Валидация: 30–80 тестовых запросов, эталонные ответы, 5–10 кейсов (штатный, аварийные, конфликты).

– Докеризация: Dockerfile, README, инструкции запуска.

3. Список источников


1. Извлечение-Augmented Generation for Knowledge-Intensive NLP Tasks (Lewis P. et al.).
2. Извлечение-Augmented Generation Survey (Gao L. et al.).
3. BM25 and Beyond (Robertson S. Et al.).
4. Attention Is All You Need (Vaswani A. et al.).
5. Assessing the Accuracy and Reliability of Large Language Models in Psychiatry Using Standardized Multiple-Choice Questions: Cross-Sectional Study (Hanss K. et al.)
6. Safety and Security Analysis of Large Language Models: Benchmarking Risk Profile and Harm Potential (Akiri C. et al.)
7. Larger and more instructable language models become less reliable (Zhou et al.)


Место выполнения практики: Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский ядерный университет «МИФИ», ИФТИС, кафедра №2 “Автоматика”.

Режим работы: понедельник, вторник: 8:30 - 18:00.

*Руководитель практики: Толоконский Андрей Олегович, к.т.н., доцент кафедры "Теплофизика"
AOTolokonskij@mephi.ru , +7-(916)-712-50-93

# Содержание

1. ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ 9
2. ВВЕДЕНИЕ 12
3. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ 14
3.1. Информационная среда АЭС 14
3.2. Типы задач 14
3.3. Основные ограничения 14
3.4 Требования безопасности и соответствия (SAFETY & COMPLIANCE) 14
3.4.1. Нормативный контекст 14
3.4.2. Классификация системы 15
3.4.3. Границы ответственности и роль системы 15
3.4.4. Ограничение влияния LLM-компонентов 16
3.4.5. Механизм deterministic fallback 17
3.4.6. Верификация и контроль корректности 17
3.4.7. Формальная валидация и тестирование 18
3.4.8. Трассируемость и аудит 18
3.4.9. Ограничения применения 19
3.4.10. Архитектурные принципы безопасности 19
4. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ 20
4.1. RAG-подход 20
4.2. Методы retrieval 20
4.3. Агентные системы 21
5. ПОСТАНОВКА ЗАДАЧИ 21
5.1. Цель 21
5.2. Требования 22
6. АРХИТЕКТУРА СИСТЕМЫ 22
6.1. Общая схема 22
6.2. Компоненты: 23
6.3. Использование PostgreSQL с расширением pgvector в качестве векторного хранилища 24

6.3.2 Расширение pgvector 24
6.3.3 Модель данных 25
6.3.4 Поиск и retrieval 25
6.3.5 Инфраструктурная интеграция 26
6.3.6 Причины выбора PostgreSQL и pgvector 26
6.3.7 Ограничения и масштабируемость 26
7. РЕАЛИЗАЦИЯ 27
7.1. Структура каталогов 27
7.2 Структура программных модулей 28
7.2 Стек 29
7.3 Pipeline данных 30
7.4. Генерация 30
7.5. Промпты и управление поведением LLM (prompt engineering) 30
7.5.1. RouterAgent – atomic/router/agent.py 30
7.5.2. CommandAgent – atomic/agents/command.py 31
7.5.3 AnalystAgent – atomic/agents/analyst.py 31
7.6. Управление зависимостями и средой выполнения (UV) 32
7.6.1. Назначение 32
7.6.2. Функциональные возможности 32
7.6.3 Использование в архитектуре системы 33
7.6.4. Роль в CI/CD и DevOps 34
7.6.5. Архитектурные преимущества 35
7.6.6. Ограничения 35
7.6.7. Перспективы развития 36
7.6.8. Роль в общей архитектуре 36
7.7. Контейнеризация и развертывание инфраструктуры сервисов проекта (DOCKER COMPOSE) 36
7.7.1. Назначение 36
7.7.2. Развёртывание базы знаний и векторного хранилища (docker-compose) 37
7.7.3. Развёртывание подсистемы мониторинга (Langfuse) 38
7.7.4. Архитектурная интеграция 40

7.7.5. Преимущества использования Docker Compose 41
7.7.6. Ограничения 41
7.7.7. Перспективы развития 42
7.7.8. Роль в архитектуре 42
8. ПОДСИСТЕМА ВЕРИФИКАЦИИ (Verifier module) 43
9. ПОДСИСТЕМА МОНИТОРИНГА И ТРЕЙСИНГА 43
9.1. Назначение подсистемы 43
9.2. Выбор технологического решения 44
9.3. Функциональные возможности 44
9.4. Место в архитектуре системы 45
9.5. Конфигурация подсистемы 47
9.6. Сценарий эксплуатации 47
9.7. Роль в обеспечении надежности системы 47
9.8. Архитектурные преимущества 48
9.9. Ограничения и направления развития 48
10. ЭКСПЕРИМЕНТАЛЬНАЯ ЧАСТЬ 49
10.1. Методика тестирования и валидации системы 49
10.1.1. Цели тестирования 49
10.1.2. Архитектура тестирования 49
10.1.3. Используемые инструменты 51
10.1.4. Режимы тестирования 51
10.1.5. Метрики качества 53
10.1.6. Структура тестового набора 54
10.1.7. Процесс выполнения тестов 54
10.1.8. Интеграция с архитектурой системы 56
10.1.9. Ограничения подхода 56
10.1.10. Перспективы развития 58
10.1.11. Роль в архитектуре 59
11. ОЦЕНКА РЕЗУЛЬТАТОВ 59
12. ОГРАНИЧЕНИЯ 60

13. ЗАКЛЮЧЕНИЕ 61
14. СПИСОК ЛИТЕРАТУРЫ 63

## 1. ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящем отчёте применяются следующие термины с соответствующими определениями:

1.1 Агент (Agent) – программный компонент системы, реализующий логику обработки пользовательского запроса, принимающий решения на основе входных данных и взаимодействующий с другими компонентами.

1.2 API (Application Programming Interface) – программный интерфейс, обеспечивающий взаимодействие между компонентами системы.

1.3 CI/CD (Continuous Integration / Continuous Deployment) – совокупность практик автоматизации процессов сборки, тестирования и развёртывания программного обеспечения.

1.4 Docker – программная платформа контейнеризации, обеспечивающая запуск приложений в изолированных средах.

1.5 Docker Compose – инструмент для декларативного описания и управления многоконтейнерными приложениями.

1.6 DSN (Data Source Name) – строка подключения к источнику данных, содержащая параметры доступа к базе данных.

1.7 Embedding (эмбеддинг) – числовое векторное представление данных, отражающее их семантическое содержание.

1.8 Judge-модель (LLM-as-a-Judge) – языковая модель, используемая для оценки качества ответов другой модели или системы на основе заданных критериев, таких как релевантность, корректность и соответствие контексту.

1.9 Langfuse – программная платформа наблюдаемости LLM-систем, обеспечивающая сбор, хранение и анализ логов, трасс и метрик.

1.10 LLM (Large Language Model) – большая языковая модель, используемая для генерации и анализа текстовых данных.

1.11 LLM-as-a-Judge – метод оценки качества результатов, при котором языковая модель используется в роли эксперта.

1.12 Lock-файл – файл, содержащий зафиксированные версии зависимостей программного проекта для обеспечения воспроизводимости среды выполнения.

1.13 Observability (наблюдаемость) – совокупность методов и инструментов, обеспечивающих мониторинг, логирование и трассировку системы.

1.14 Pipeline (пайплайн) – последовательность этапов обработки данных или запроса в системе.

1.15 PostgreSQL – объектно-реляционная система управления базами данных.

1.16 pgvector – расширение СУБД PostgreSQL, обеспечивающее хранение и обработку векторных данных.

1.17 pyproject.toml – конфигурационный файл Python-проекта, содержащий описание зависимостей и параметров сборки.

1.18 RAG (Retrieval-Augmented Generation) – метод генерации ответов, основанный на объединении языковой модели и механизма извлечения внешних данных.

1.19 Retriever (ретривер) – компонент системы, выполняющий поиск релевантной информации на основе входного запроса.

1.20 Router (маршрутизатор) – компонент системы, определяющий сценарий обработки запроса и направляющий его в соответствующий модуль.

1.21 Tracing (трейсинг) – процесс отслеживания прохождения запроса через компоненты системы с фиксацией промежуточных состояний.
1.22 SLI (Service Level Indicator) – измеримая метрика качества предоставления сервиса, отражающая фактическое состояние работы системы (например, latency, error rate, availability) и используемая для объективной оценки уровня обслуживания.

1.23 SLO (Service Level Objective) – целевое значение или диапазон для SLI, определяющее требуемый уровень качества сервиса с точки зрения инженерных и эксплуатационных требований.

1.24 SLA (Service Level Agreement) – формализованное соглашение об уровне обслуживания между поставщиком и потребителем сервиса, включающее целевые SLO и условия ответственности за их нарушение.

1.25 uv – инструмент управления зависимостями и запуска Python-приложений, обеспечивающий воспроизводимость среды выполнения.

1.26 Vector Store (векторное хранилище) – подсистема хранения и поиска данных, представленных в виде векторных эмбеддингов.

1.27 Доменно-ограниченная среда – область применения системы, характеризующаяся наличием чётко определённого набора предметных знаний, терминологии, правил и источников данных, в рамках которых формируются и обрабатываются запросы, что позволяет ограничить вариативность интерпретаций и повысить точность и достоверность результатов работы системы.

1.28 Интерпретируемость – свойство системы, заключающееся в возможности объяснения логики формирования её результатов, включая используемые данные, промежуточные шаги обработки и принятые решения, в форме, понятной человеку.

1.29 Human-in-the-loop – подход к построению систем, при котором человек включён в контур обработки и принятия решений, осуществляя контроль, валидацию и при необходимости корректировку результатов, формируемых автоматизированными компонентами системы.

1.30 Hallucinations (галлюцинации) – это случаи, когда модель «придумывает» факты, детали или даже источники, хотя на самом деле их нет.

## 2. ВВЕДЕНИЕ

Эксплуатация атомных электростанций (АЭС) требует высокой точности и надежности при принятии решений. Оператор работает в условиях значительного объема нормативно-технической документации, включающей регламенты, инструкции и аварийные процедуры.

Основная проблема заключается в том, что доступ к необходимой информации:

- занимает значительное время;
- требует высокой квалификации;
- сопряжен с риском ошибок.

Современные методы искусственного интеллекта позволяют создавать системы информационной поддержки оператора, обеспечивающие доступ к релевантным данным и источникам без участия в процессе принятия решений

нового поколения, способные обрабатывать большие объемы слабоструктурированной информации и предоставлять оператору контекстно-зависимую информацию с указанием источников.

Ключевой проблемой применения больших языковых моделей (LLM) в критически важных системах является отсутствие гарантированной достоверности, а также невозможность явной трассировки источников информации. Это ограничивает их применение в областях с повышенными требованиями к надежности, таких как атомная энергетика.

Одним из наиболее перспективных подходов, позволяющих преодолеть указанные ограничения, является Retrieval-Augmented Generation (RAG). Данный подход основан на разделении функций хранения знаний и генерации ответа: фактическая информация извлекается из внешней базы знаний, после чего используется языковой моделью для формирования ответа.

В отличие от альтернативного подхода, основанного на дообучении моделей (fine-tuning), при котором знания имплицитно встраиваются в параметры модели, RAG обеспечивает:

- явную привязку ответа к источникам (citation-based generation);
- возможность верификации и аудита;
- актуализацию знаний без переобучения модели;
- контроль используемой информации на уровне инфраструктуры.

Это делает RAG архитектурно предпочтительным решением для систем поддержки принятия решений в условиях повышенных требований к безопасности и надежности.

Целью данной работы является разработка прототипа интеллектуального ассистента, обеспечивающего:

- быстрый поиск информации;

- генерацию ответов, основанных на извлечённых фрагментах документов, с обязательным указанием источников и без формирования рекомендаций;
- автоматическую верификацию.

## 3. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

### 3.1. Информационная среда АЭС

Оператор взаимодействует со следующими типами документов:

- эксплуатационные инструкции;
- аварийные процедуры;
- технические спецификации;
- справочные таблицы.

### 3.2. Типы задач

1. Поиск информации
2. Выполнение процедур
3. Диагностика неисправностей
4. Работа в аварийных режимах

### 3.3. Основные ограничения

- критичность ошибок;
- необходимость трассируемости решений;
- требования к интерпретируемости.

### 3.4 Требования безопасности и соответствия (SAFETY & COMPLIANCE)

#### 3.4.1. Нормативный контекст

Разработка систем, применяемых в атомной энергетике, осуществляется с учётом требований функциональной безопасности и надёжности программного обеспечения. К числу базовых нормативных ориентиров относятся:

- IEC 61508 (ГОСТ IEC 61508) - функциональная безопасность электрических, электронных и программируемых электронных систем;
- IEC 60880 (ГОСТ Р МЭК 60880-2010) - программное обеспечение систем, важных для безопасности атомных электростанций;
- отраслевые стандарты и регламенты, определяющие требования к надёжности, верификации и прослеживаемости.

В рамках настоящей работы разрабатываемая система рассматривается как вспомогательная информационная система, не выполняющая функций безопасности и не входящая в контур управления технологическими процессами.

### 3.4.2. Классификация системы

Система классифицируется как:

- non-safety system – не выполняет функций защиты или управления;
- decision support (informational) – обеспечивает доступ к информации, но не участвует в принятии решений;
- offline/adjacent system – функционирует вне контура реального управления оборудованием.

Система не оказывает прямого воздействия на технологические процессы и не может инициировать управляющие воздействия.

### 3.4.3. Границы ответственности и роль системы

В архитектуре системы вводится строгое разграничение ответственности:

- система предоставляет релевантную информацию и ссылки на нормативно-технические источники;
- система не формирует предписаний, рекомендаций или указаний к действию;
- система не участвует в процессе принятия решений оператором;
- система не интерпретирует данные в форме, предполагающей выбор конкретного действия;
- ответственность за интерпретацию информации и принятие решений полностью лежит на операторе.

Таким образом, система выполняет роль информационного посредника, а не участника процесса управления.

### 3.4.4. Ограничение влияния LLM-компонентов

Используемые языковые модели (LLM) применяются исключительно для:

- генерации текстового представления извлечённой информации;
- структурирования и суммаризации контента;
- обеспечения удобства взаимодействия с пользователем.

При этом вводятся следующие ограничения:

- запрещено формирование рекомендаций и предписывающих формулировок;
- запрещено использование модели как источника истины;
- все ответы должны быть привязаны к извлечённым источникам;
- при отсутствии достаточного контекста генерация ответа блокируется.

LLM-компонент рассматривается как инструмент представления информации, а не как механизм принятия решений!

### 3.4.5. Механизм deterministic fallback

Для обеспечения надёжности работы системы реализуется механизм отказоустойчивого поведения (deterministic fallback):

В случае:

- отсутствия релевантных данных;
- низкой уверенности в результате retrieval;
- конфликтов между источниками;
- невозможности верификации ответа;

система:

- не формирует ответ на основе генерации;
- уведомляет пользователя о недостаточности данных;
- предоставляет ссылки на исходные документы;
- предлагает уточнение запроса.

Данный подход реализует принцип: отсутствие ответа предпочтительнее недостоверного ответа.

### 3.4.6. Верификация и контроль корректности

Подсистема верификации обеспечивает дополнительный уровень контроля качества и включает:

- проверку наличия и корректности источников;
- сопоставление ответа с извлечённым контекстом (faithfulness);
- выявление логических противоречий;
- проверку числовых и фактологических данных (при наличии формализуемых правил).

Верификация реализуется как комбинация:

- rule-based проверок (детерминированные правила);
- семантического сопоставления;
- cross-check между источниками.

Результаты верификации могут использоваться для блокировки или модификации ответа.

### 3.4.7. Формальная валидация и тестирование

Тестирование системы дополняется элементами формальной валидации:

- формализация требований к системе;
- построение трассируемости (requirement → test case);
- выделение детерминированных сценариев проверки;
- проверка граничных условий (edge cases);
- контроль регрессий при изменении компонентов.

LLM-оценка (LLM-as-a-Judge) используется как вспомогательный инструмент и не является единственным критерием качества.

### 3.4.8. Трассируемость и аудит

Система обеспечивает полную трассируемость обработки запросов:

- фиксируется входной запрос;
- сохраняются извлечённые документы;
- логируются промежуточные шаги pipeline;
- сохраняется финальный ответ;
- обеспечивается возможность восстановления цепочки обработки (trace).

Это позволяет:

- проводить аудит работы системы;

- анализировать ошибки;
- обеспечивать соответствие требованиям прозрачности и воспроизводимости.

### 3.4.9. Ограничения применения

Несмотря на реализованные механизмы контроля, система имеет ограничения:

- зависимость от качества и полноты базы знаний;
- вероятность ошибок генерации, обусловленная природой LLM;
- ограниченная интерпретируемость сложных семантических выводов;
- невозможность использования в качестве источника принятия решений.

Система предназначена исключительно для информационной поддержки, а не для управления или автоматизации действий оператора.

### 3.4.10. Архитектурные принципы безопасности

В основе системы лежат следующие принципы:

- Separation of Concerns – разделение функций поиска, генерации и верификации;
- Human-in-the-loop – обязательное участие человека в принятии решений;
- Fail-safe behavior – безопасное поведение при ошибках (fallback);
- Traceability by design – встроенная трассируемость всех операций;
- Non-decision AI – исключение участия AI в принятии решений.

# 4. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ

## 4.1. RAG-подход

Retrieval (поиск)

Извлечение релевантных фрагментов из внешней базы знаний (векторное хранилище, документы) на основе запроса пользователя.

Generation (генерация)

Формирование ответа языковой моделью с использованием найденного контекста.

Преимущества:

- Уменьшение hallucinations

Модель опирается на реальные документы, а не только на внутренние знания.

- Использование внешней базы знаний

Позволяет работать с актуальными и доменными данными без переобучения модели.

- Возможность ссылочной проверки

Каждый ответ может быть сопоставлен с конкретным источником (traceability).

## 4.2. Методы retrieval

Dense retrieval

- Embeddings – представление текста в виде векторов в многомерном пространстве.
- Semantic search

Поиск по смысловой близости, а не по точному совпадению слов.

Sparse retrieval

- BM25

Классический алгоритм поиска, основанный на частоте слов и их значимости в документе.

Hybrid retrieval

- Комбинированный подход

Объединение dense и sparse методов для повышения точности поиска (semantic + keyword match).

### 4.3. Агентные системы

Router - определяет тип запроса и выбирает сценарий обработки (поиск, анализ, выполнение).

Retriever - извлекает релевантные данные из базы знаний.

Generator - формирует ответ на основе извлеченного контекста.

Verifier - проверяет корректность ответа (источники, логика, числовые значения).

Преимущество – масштабируемость и управляемость

Функции разделены между независимыми компонентами, что упрощает развитие системы, тестирование и контроль качества.

## 5. ПОСТАНОВКА ЗАДАЧИ

### 5.1. Цель

Создание прототипа RAG-системы для операторов АЭС.

### 5.2. Требования

- Достоверность – ответы системы основаны на проверяемых источниках и сопровождаются ссылками, позволяющими подтвердить корректность информации.
- Трассируемость – возможность отследить полный путь обработки запроса: от входных данных до итогового ответа и использованных источников.
- Модульность – система состоит из независимых компонентов с чёткими интерфейсами, которые можно развивать и заменять без влияния на остальные части.
- Безопасность – защита данных и системы от несанкционированного доступа, ошибок и вредоносного использования.
- Защита от prompt injection – это способность системы предотвращать выполнение вредоносных или манипулятивных инструкций, внедрённых в пользовательский ввод или внешние данные.

## 6. АРХИТЕКТУРА СИСТЕМЫ

### 6.1. Общая схема

Pipeline:

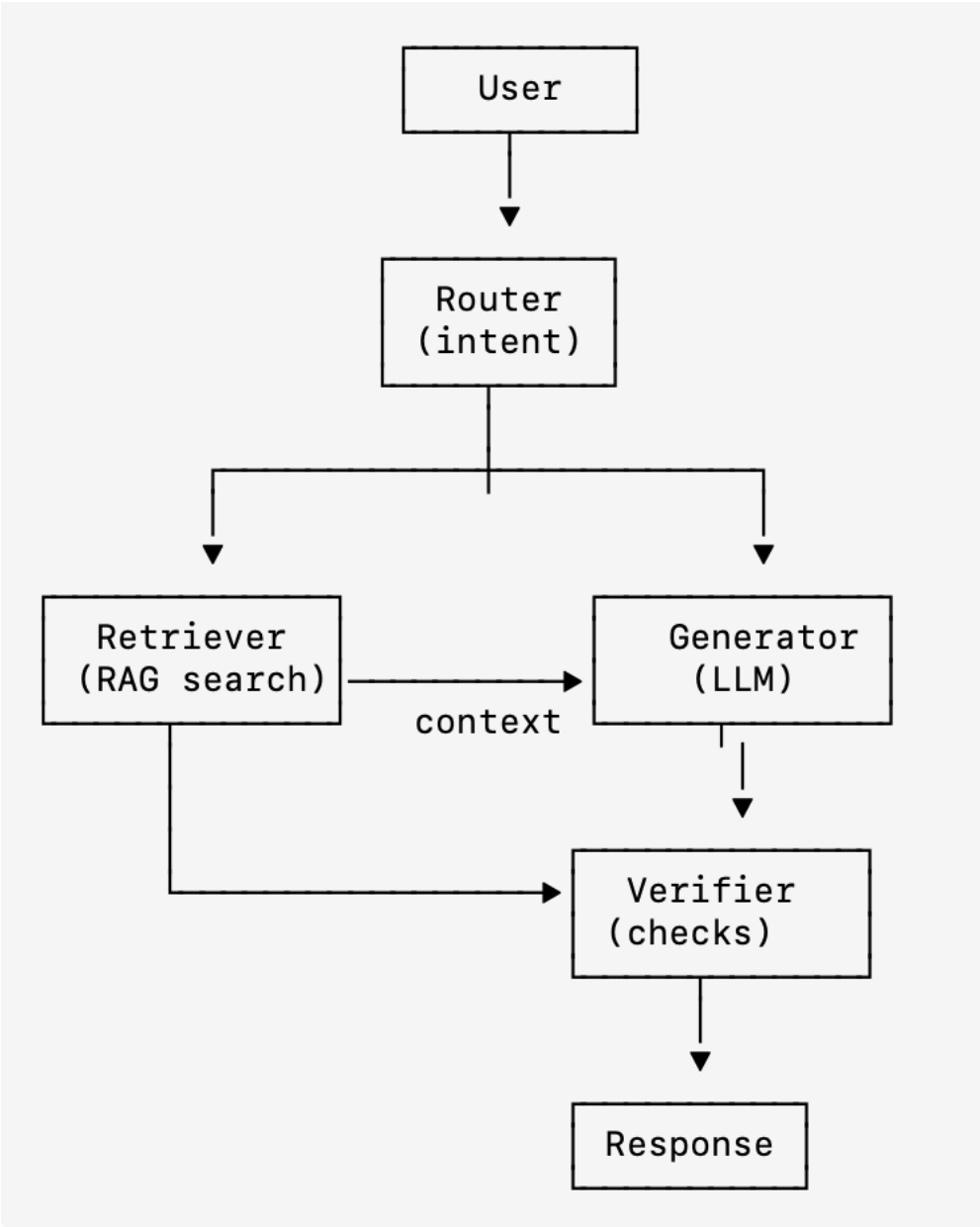



Рис. 1. Pipeline

**6.2. Компоненты:**

- API - FastAPI
- Router - intent detection
- Retriever:
  - vector DB
  - BM25 fallback

- Generator - LLM
- Verifier - проверки
- Memory - контекст

### 6.3. Использование PostgreSQL с расширением pgvector в качестве векторного хранилища

6.3.1 Назначение и роль в архитектуре

В рамках разрабатываемой системы подсистема хранения знаний реализована на базе СУБД PostgreSQL с использованием расширения pgvector. Данное решение обеспечивает хранение векторных представлений текстовых данных (эмбеддингов) и выполнение семантического поиска, необходимого для реализации подхода RAG (см. 1.17).

PostgreSQL выступает в роли универсального хранилища, объединяющего:

- структурированные данные (метаданные документов);
- неструктурированные данные (тексты);
- векторные представления (эмбеддинги).

Это позволяет избежать внедрения отдельной специализированной векторной базы данных и снизить архитектурную сложность системы.

#### 6.3.2 Расширение pgvector

Расширение pgvector добавляет в PostgreSQL поддержку векторных типов данных и операций над ними, включая:

- хранение векторов фиксированной размерности;
- вычисление расстояний (cosine, L2, inner product);
- индексацию для ускорения поиска ближайших соседей.

Использование pgvector позволяет реализовать семантический поиск непосредственно на уровне СУБД, без вынесения этой логики в отдельный сервис.

### 6.3.3 Модель данных

Таблицы для хранения документов и их эмбеддингов включает следующие поля:

- id – уникальный идентификатор записи;
- content – текст документа или его фрагмента;
- embedding – векторное представление текста;
- metadata – дополнительные атрибуты (источник, теги и др.).

Такой подход обеспечивает:

- связность данных (текст + вектор + метаданные);
- удобство фильтрации и постобработки;
- расширяемость модели без изменения архитектуры.

### 6.3.4 Поиск и retrieval

Поиск релевантных документов осуществляется с использованием операций сравнения векторов, например:

- cosine similarity – для оценки семантической близости;
- L2 distance – для метрических задач.

Типовой сценарий:

1. Пользовательский запрос преобразуется в эмбеддинг.
2. Выполняется поиск ближайших векторов в базе данных.
3. Возвращаются top-k релевантных документов.
4. Результаты передаются в LLM для генерации ответа.

Таким образом, PostgreSQL с pgvector реализует слой Retriever (см. 1.18) в пайплайне системы.

### 6.3.5 Инфраструктурная интеграция

Развёртывание PostgreSQL с поддержкой pgvector осуществляется с использованием docker-compose (см. 1.5), что обеспечивает:

- автоматическую настройку окружения;
- воспроизводимость конфигурации;
- быстрый запуск системы на новых узлах.

Подключение к базе данных выполняется через строку DSN (см. 3.6), передаваемую через переменные окружения.

### 6.3.6 Причины выбора PostgreSQL и pgvector

Выбор данного решения обусловлен следующими факторами:

- Снижение архитектурной сложности - отсутствие необходимости внедрения отдельной векторной БД (например, FAISS, Milvus).
- Консистентность данных - хранение текста, метаданных и эмбеддингов в одной транзакционной системе.
- Интеграция с существующим стеком - PostgreSQL широко используется в корпоративных системах.
- Гибкость и расширяемость - возможность добавления новых индексов, фильтров и бизнес-логики на уровне SQL.

### 6.3.7 Ограничения и масштабируемость

Несмотря на преимущества, использование PostgreSQL с pgvector имеет ограничения:

- более низкая производительность по сравнению с специализированными векторными БД при больших объёмах данных;
- ограниченные возможности распределённого масштабирования;
- необходимость настройки индексов (например, IVFFlat) для повышения производительности.

В долгосрочной перспективе (при росте объёма данных и нагрузки) возможно:

- выделение retrieval-слоя в отдельный сервис;
- переход на специализированные решения (например, распределённые vector DB);
- внедрение гибридного поиска (BM25 + embeddings).

Использование PostgreSQL с расширением pgvector представляет собой сбалансированное архитектурное решение для прототипа и систем среднего масштаба, обеспечивающее:

- простоту внедрения;
- воспроизводимость инфраструктуры;
- достаточную производительность для задач RAG.

Данный подход позволяет сосредоточиться на развитии логики интеллектуального ассистента, откладывая усложнение инфраструктуры до этапа масштабирования системы.

## 7. РЕАЛИЗАЦИЯ

### 7.1. Структура каталогов

```
atomic/
    ├── main.py              # CLI / API точка входа
    ├── atomic/
```

```
│   ├── api/server.py       # FastAPI: POST /chat, GET /health
│   ├── router/agent.py     # Роутер
│   ├── agents/             # retriever, command, analyst
│   ├── memory/             # chat, pgvector, vector (in-memory)
│   ├── tools/code_executor.py
│   ├── llm/client.py       # Ollama chat
│   ├── embeddings/client.py
│   ├── observability/langfuse.py
│   └── orchestration/workflow.py
├── scripts/load_dataset.py # Загрузка dataset/*.txt в RAG
├── docker-compose.yml      # PostgreSQL + pgvector
└── config.py               # Модели, DSN, Langfuse
```

### 7.2 Структура программных модулей

Реализованный проект atomic оформлен как Python-пакет atomic/ с двумя точками входа в корне репозитория: сценарий CLI и HTTP API на FastAPI (main.py). Общая логика диалога сосредоточена в классе AtomicWorkflow: он получает запрос пользователя, при необходимости инициализирует демонстрационные данные в векторной памяти и направляет обработку по цепочке специализированных агентов.

Конфигурация централизована в модуле config.py (модели Ollama, параметры подключения к PostgreSQL/pgvector, флаги использования векторного хранилища и опционального трейсинга Langfuse). К нему обращаются слои работы с языковыми моделями, эмбеддингами, памятью и агентами.

Маршрутизация намерений выполняется RouterAgent (router/agent.py): с помощью LLM запрос относится к одному из типов – поиск по знаниям (retrieval), выполнение кода (command), анализ/рассуждение (analysis) или общий диалог (general). Далее AtomicWorkflow выбирает подходящего агента из

набора RetrieverAgent, CommandAgent и AnalystAgent (модуль agents/), опираясь на единый контракт BaseAgent.

Память включает кратковременную историю диалога (ChatMemory) и векторное хранилище: либо in-memory реализация для демонстрации, либо PostgreSQL с расширением pgvector через PgVectorStore и PgVectorMemory; переключение задаётся фабрикой create_vector_memory(). Поиск по базе знаний в Retriever-агенте опирается на эмбеддинги (EmbeddingClient) и вызовы LLM (LLMClient), оба модуля обращаются к Ollama. Выполнение сгенерированного кода изолировано в CodeExecutor (tools/code_executor.py).

Наблюдаемость (опционально) реализована в observability/langfuse.py: трассировка запросов workflow и обращений к LLM/эмбеддингам при наличии ключей Langfuse. Вспомогательные артефакты репозитория – скрипт загрузки датасета, Docker Compose для БД и Langfuse, каталог тестов на pytest – не входят в пакет atomic, но поддерживают развёртывание и проверку качества.

### 7.2 Стек

- Python – основной язык реализации системы, используется для оркестрации pipeline, интеграции компонентов и разработки агентной логики.
- PostgreSQL + pgvector – СУБД с поддержкой векторных данных; используется для хранения текстов, метаданных и эмбеддингов, а также выполнения семантического поиска.
- FAISS – библиотека высокопроизводительного поиска ближайших соседей; применяется для ускорения retrieval и работы с большими объёмами векторных данных.
- Ollama – платформа локального запуска LLM; обеспечивает генерацию ответов без зависимости от внешних API и контроль над моделью.

### 7.3 Pipeline данных

1. Парсинг

Извлечение текстовых данных из исходных документов (инструкции, регламенты и т.д.) и приведение к унифицированному формату.

2. Chunking

Разбиение текста на логические фрагменты (чанки) фиксированного или адаптивного размера для повышения точности поиска и генерации.

3. Embeddings

Преобразование текстовых фрагментов в векторное представление, отражающее их семантический смысл.

4. Индексация

Сохранение эмбеддингов и связанных данных в хранилище (pgvector/FAISS) с построением индексов для быстрого поиска релевантных фрагментов.

### 7.4. Генерация

- prompt engineering
- обязательные ссылки

Ниже – оформленная глава, согласованная по стилю с документом и логически встраиваемая в раздел 6. Реализация (например, как 6.4) или как отдельный инфраструктурный подраздел.

### 7.5. Промпты и управление поведением LLM (prompt engineering)

Критически важная часть логики системы. Список промптов в проекте:

#### 7.5.1. RouterAgent – atomic/router/agent.py

ROUTER_PROMPT - определение intent:

*"""Определи намерение пользователя по запросу. Ответь ОДНИМ словом: retrieval, command, analysis или general.*

*retrieval – поиск в базе знаний, инструкций, документации. Примеры: "как запустить", "найди информацию", "что известно о", "инструкция по"*
*command – выполнение КОДА на Python/bash. Только если явно просят выполнить код: "выполни код", "запусти код", "run: print(1+1)"*
*analysis – анализ, планирование, объяснения: "проанализируй", "объясни", "спланируй"*
*general – общий вопрос, разговор*

*Важно: "Как запустить систему?" = retrieval (инструкции). "Выполни код print(1)" = command.*

*Запрос: {query}*

*Ответ (одно слово):"""*

**7.5.2. CommandAgent – atomic/agents/command.py**

CODE_GEN_PROMPT – генерация Python-кода:

*"""Пользователь просит выполнить код или действие. Сгенерируй ТОЛЬКО код на Python, без пояснений и markdown.*
*Код должен быть самодостаточным, с print() для вывода результата.*
*Если запрос неясен – сгенерируй print("Уточните, какой код выполнить").*
*Запрос: {query}*
*Код (только Python):"""*

**7.5.3 AnalystAgent – atomic/agents/analyst.py**

ANALYST_PROMPT – анализ и ответ:

*"""Ты – аналитик и эксперт. Ответь на вопрос пользователя: рассуждай, анализируй, давай рекомендации.*
*Будь конкретным и полезным. Отвечай на том же языке, что и вопрос.*
*Вопрос: {query}*
*Ответ:*
*Примечание: AnalystAgent добавляет к промпту блок Предыдущий контекст: \n{history_text}\n\n, если есть история чата (последние 4 сообщения)."""*

## 7.6. Управление зависимостями и средой выполнения (UV)

### 7.6.1. Назначение

В рамках проекта используется менеджер зависимостей и среды выполнения uv, обеспечивающий воспроизводимость окружения, унификацию запуска приложений и стандартизацию процессов разработки.

Использование единого инструмента управления зависимостями является критически важным для:

- обеспечения консистентности среды между разработкой и эксплуатацией;
- снижения рисков несовместимости библиотек;
- упрощения деплоя и тестирования;
- повышения управляемости инфраструктуры проекта.

### 7.6.2. Функциональные возможности

Менеджер uv выполняет следующие функции:

Управление зависимостями

- установка библиотек на основе файла pyproject.toml;
- автоматическое разрешение зависимостей;
- формирование lock-файла uv.lock.

Lock-файл фиксирует точные версии всех зависимостей, что обеспечивает:

- воспроизводимость окружения;
- идентичность среды на всех этапах жизненного цикла;
- предсказуемость поведения системы.

Управление средой выполнения

- изоляция окружения проекта;
- устранение конфликтов между зависимостями;
- упрощение локальной и серверной настройки.

В отличие от традиционных подходов, uv минимизирует накладные расходы на управление виртуальными окружениями.

Унификация запуска

uv используется как единая точка входа для запуска всех компонентов системы:

- основное приложение;
- API-сервер;
- служебные скрипты;
- тесты.

Пример запуска:

uv run python main.py

### 7.6.3 Использование в архитектуре системы

Инструмент uv выступает как единая точка управления запуском и средой выполнения всех процессов системы, обеспечивая воспроизводимость и стандартизацию.

Запуск приложения

- запуск backend-компонентов – uv управляет зависимостями и окружением, гарантируя корректный старт всех модулей системы (API, агенты, orchestration).
- выполнение pipeline обработки запросов – обеспечивает единообразный запуск логики обработки (Router → Retriever → Generator → Verifier) без расхождений между средами.
- запуск сервисов API – стандартизирует старт FastAPI-сервера и связанных сервисов через единый runtime.

Обработка данных

- запуск скриптов парсинга и загрузки датасета – uv обеспечивает воспроизводимый запуск ETL-процессов и загрузки данных в RAG-хранилище.
- выполнение preprocessing и индексации – гарантирует консистентность этапов chunking, embeddings и записи в векторное хранилище.

Тестирование

- запуск модульных тестов (pytest) – обеспечивает единое тестовое окружение и предсказуемость результатов.
- выполнение LLM-оценок (DeepEval) – позволяет запускать probabilistic/semantic тесты в контролируемой среде.
- поддержка fast/full режимов – унифицирует запуск разных профилей тестирования (быстрые проверки и полная оценка качества).

Пример:

```
uv run pytest tests/test_atomic.py -v
```

**7.6.4. Роль в CI/CD и DevOps**

Использование uv формирует основу для стандартизации процессов CI/CD:

- единый способ инициализации окружения;
- воспроизводимый билд;
- сокращение времени развертывания;
- упрощение контейнеризации (Docker).

uv может быть напрямую интегрирован в pipeline:

- сборка окружения через uv sync;
- запуск тестов через uv run;
- контроль версий зависимостей через uv.lock.

### 7.6.5. Архитектурные преимущества

Использование uv соответствует современным принципам проектирования:

- Reproducibility by design – гарантированная воспроизводимость окружения;
- Single entry point – единая точка запуска;
- Infrastructure simplification – снижение сложности DevOps-слоя;
- Environment consistency – единое поведение системы во всех средах.

### 7.6.6. Ограничения

К ограничениям подхода можно отнести:

- зависимость от конкретного инструмента;
- необходимость включения uv в инфраструктурные стандарты проекта;
- ограниченная распространенность по сравнению с классическими решениями (pip, poetry).

### 7.6.7. Перспективы развития

Дальнейшее развитие использования uv в проекте включает:

- интеграцию с CI/CD пайплайнами;
- использование в production-среде;
- автоматизацию сборки контейнеров;
- внедрение политики управления зависимостями (dependency governance);
- контроль уязвимостей зависимостей.

### 7.6.8. Роль в общей архитектуре

uv выступает как базовый инфраструктурный компонент, обеспечивающий:

- воспроизводимость системы;
- управляемость зависимостей;
- стандартизацию запуска и тестирования.

В долгосрочной перспективе он является частью платформенного слоя (platform layer), обеспечивающего устойчивость и масштабируемость разработки, что особенно важно для систем, использующих LLM и сложные пайплайны обработки данных.

Ниже – цельная, исправленная глава с учётом архитектурного разделения Langfuse как внешнего сервиса.

## 7.7. Контейнеризация и развертывание инфраструктуры сервисов проекта (DOCKER COMPOSE)

### 7.7.1. Назначение

Для управления внешними инфраструктурными зависимостями в проекте используется Docker Compose, обеспечивающий быстрое и воспроизводимое развёртывание необходимых сервисов.

Контейнеризация применяется для:

- развертывания СУБД и вспомогательных компонентов;
- стандартизации окружения разработки и тестирования;
- снижения объёма ручной настройки;
- обеспечения переносимости системы.

Инфраструктура описывается декларативно, что соответствует подходу Infrastructure as Code (IaC) и упрощает сопровождение системы.

### 7.7.2. Развёртывание базы знаний и векторного хранилища (docker-compose)

Цель – обеспечить единый воспроизводимый контур, который запускается одной командой и сразу поднимает всю необходимую инфраструктуру без ручной настройки, включая базу данных и сопутствующие сервисы. Система должна предоставлять готовое хранилище для текстовых данных и эмбеддингов, а также стандартизированную среду, упрощающую разработку, тестирование и последующее развертывание, снижая операционную сложность и риски расхождения конфигураций между средами.

Файл docker-compose.yml, расположенный в корне проекта, описывает конфигурацию сервиса PostgreSQL с расширением pgvector, используемого в качестве векторного хранилища.

Данный компонент является частью RAG-подсистемы и обеспечивает:

- хранение эмбеддингов;
- выполнение семантического поиска;

- поддержку retrieval-операций.

Развёртывание

Запуск осуществляется одной командой:

*docker compose up -d*

После запуска контейнера приложение подключается к базе данных через строку:

*ATOMIC_PG_VECTOR_DSN=«postgresql://atomic:atomic@localhost:5432/atomic"*

С архитектурной точки зрения PostgreSQL с pgvector выполняет роль унифицированного слоя хранения и поиска, объединяя функции persistence layer и retrieval backend. Это позволяет централизовать управление знаниями системы и обеспечить консистентность работы с векторными представлениями.

Использование Docker Compose формирует декларативную модель инфраструктуры (Infrastructure as Code), обеспечивая автоматическое развертывание сервисов, управление конфигурацией окружения и сетевыми параметрами, а также изоляцию зависимостей.

Ключевые преимущества подхода заключаются в воспроизводимости окружения, сокращении времени запуска системы, минимизации ручных операций, снижении вероятности конфигурационных расхождений и упрощении процесса онбординга новых участников проекта.

### 7.7.3. Развёртывание подсистемы мониторинга (Langfuse)

Назначение

В подкаталоге langfuse/ расположен отдельный файл docker-compose.yml, предназначенный для развёртывания платформы Langfuse, используемой в системе для мониторинга и трейсинга.

Langfuse выполняет функции:

- сбора логов и трасс выполнения;
- анализа качества ответов;
- визуализации работы pipeline.

Причины архитектурного разделения

Конфигурация Langfuse вынесена в отдельный docker-compose стек, поскольку он рассматривается как внешний инфраструктурный сервис.

Такое решение обусловлено следующими факторами:

- слабая связанность (decoupling)

Langfuse не входит в состав core-системы и может использоваться независимо;

- переиспользуемость (reusability)

один экземпляр Langfuse может обслуживать несколько проектов;

- независимый жизненный цикл (independent lifecycle)

обновление и масштабирование системы мониторинга не влияет на основное приложение;

- разделение инфраструктурных доменов

слой хранения (PostgreSQL) и слой наблюдаемости (Langfuse) управляются отдельно.

Развёртывание

Локальный экземпляр Langfuse запускается командой:

*docker compose -f langfuse/docker-compose.yml up -d*

После запуска приложение подключается к сервису через переменные окружения:

- LANGFUSE_BASE_URL – базовый URL экземпляра Langfuse, используемого для трассировки, логирования и наблюдаемости LLM-запросов. Определяет endpoint, к которому подключается приложение для отправки telemetry-данных.
- LANGFUSE_SECRET_KEY – секретный ключ сервисного доступа к Langfuse. Используется серверной частью приложения для аутентификации при записи событий, трассировок и метрик. Должен храниться только на стороне backend и не попадать в клиентские окружения.
- LANGFUSE_PUBLIC_KEY – публичный идентификатор проекта в Langfuse. Применяется для идентификации источника данных при инициализации SDK и связывания телеметрии с конкретным проектом. Может использоваться в более широком контексте, включая клиентские среды при необходимости.

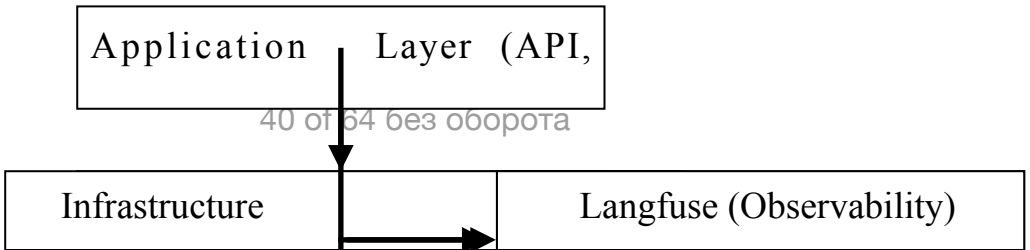
Практическое значение

Выделение Langfuse в отдельный стек позволяет:

- использовать как локальный, так и облачный экземпляр;
- подключаться к корпоративной observability-платформе;
- избежать дублирования инфраструктуры;
- упростить сопровождение и обновление.

### 7.7.4. Архитектурная интеграция

Docker Compose реализует слой управления внешними сервисами и интегрируется в архитектуру следующим образом:




40 of 64 без оборота

Рис. 2. Архитектура интеграции

Данное разделение архитектуры соответствует ключевым инженерным принципам построения устойчивых систем. Принцип Separation of Concerns реализуется через чёткое разграничение ответственности между компонентами (инфраструктура, хранилище, прикладная логика, observability), что упрощает сопровождение и эволюцию системы. Composable Architecture достигается за счёт модульности и независимости сервисов, позволяя гибко изменять, масштабировать и переиспользовать отдельные части без влияния на систему в целом. Environment Parity обеспечивается использованием контейнеризации и декларативного описания инфраструктуры, что гарантирует идентичность сред разработки, тестирования и промышленной эксплуатации, снижая риски, связанные с расхождениями конфигураций.

### 7.7.5. Преимущества использования Docker Compose

Применение Docker Compose обеспечивает:

- развёртывание системы одной командой;
- воспроизводимость среды;
- снижение порога входа;
- стандартизацию инфраструктуры;
- удобство локальной разработки и тестирования.

### 7.7.6. Ограничения

К ограничениям подхода относятся:

- ограниченные возможности масштабирования;
- отсутствие встроенной оркестрации;
- зависимость от Docker-окружения;
- дополнительные накладные расходы на ресурсы.

### 7.7.7. Перспективы развития

Дальнейшее развитие инфраструктурного слоя направлено на повышение масштабируемости, управляемости и отказоустойчивости системы. В production-среде целесообразен переход к оркестрации на базе Kubernetes, обеспечивающей автоматическое управление жизненным циклом сервисов и горизонтальное масштабирование. Параллельно возможно использование managed-сервисов (DBaaS, observability-платформ), позволяющих снизить операционную нагрузку и сфокусироваться на прикладной логике. Внедрение CI/CD pipeline обеспечивает автоматизацию сборки, тестирования и доставки изменений, сокращая time-to-market и повышая стабильность релизов. Развитие GitOps-подхода позволяет сделать конфигурацию и состояние инфраструктуры управляемыми через версионируемые репозитории, обеспечивая прозрачность и воспроизводимость изменений. Дополнительно, централизованное управление конфигурацией формирует единый контроль над параметрами среды и секретами, повышая безопасность и согласованность работы всех компонентов системы.

### 7.7.8. Роль в архитектуре

Docker Compose выступает как базовый механизм управления инфраструктурой на этапе разработки и прототипирования.

Он обеспечивает:

- воспроизводимость системы;
- изоляцию зависимостей;
- стандартизацию развёртывания.

В дальнейшем данный подход может быть эволюционирован в полноценную платформенную инфраструктуру, обеспечивающую

масштабируемость, отказоустойчивость и управляемость системы на уровне промышленной эксплуатации.

## 8. ПОДСИСТЕМА ВЕРИФИКАЦИИ (Verifier module)

Функции:

- проверка источников;
- проверка чисел;
- выявление конфликтов.

Типы проверок:

- rule-based;
- semantic;
- cross-check.

## 9. ПОДСИСТЕМА МОНИТОРИНГА И ТРЕЙСИНГА

### 9.1. Назначение подсистемы

В рамках разработки интерактивного ассистента для операторов атомных электростанций реализована подсистема мониторинга и трейсинга, обеспечивающая наблюдаемость работы агентной RAG-системы.

Ключевой задачей данной подсистемы является фиксация полного жизненного цикла обработки пользовательского запроса, включая промежуточные этапы работы агентов, обращения к внешним сервисам и формирование итогового ответа.

В отличие от традиционного логирования, трейсинг ориентирован на восстановление причинно-следственных связей между компонентами системы,

что особенно важно в условиях повышенных требований к надежности и трассируемости.

### 9.2. Выбор технологического решения

В качестве платформы мониторинга выбрана Langfuse, специализированная система наблюдаемости для приложений на базе больших языковых моделей.

Выбор данного решения обусловлен следующими факторами:

- нативная поддержка LLM-приложений и агентных архитектур;
- возможность трейсинга цепочек вызовов (chains, agents, tools);
- встроенные механизмы анализа качества ответов;
- удобный веб-интерфейс для визуализации метрик и логов.

Таким образом, Langfuse выступает как центральный компонент observability-слоя системы.

### 9.3. Функциональные возможности

Подсистема мониторинга и трейсинга реализует следующие функции:

Сбор и агрегация данных

- логирование входящих запросов и сформированных ответов;
- фиксация промежуточных шагов обработки;
- регистрация параметров вызовов LLM и retrieval-подсистемы.

Трейсинг выполнения

- построение цепочек вызовов агентов (trace);
- измерение времени выполнения отдельных этапов;
- выявление узких мест (bottlenecks).

Анализ качества

- оценка релевантности ответов;
- выявление ошибок генерации;
- анализ конфликтов источников.

Визуализация

- графическое представление трейсингов;
- метрики задержек и ошибок;
- статистика использования компонентов.

## 9.4. Место в архитектуре системы

Подсистема трейсинга интегрирована в ключевые точки обработки запроса и охватывает весь pipeline системы:

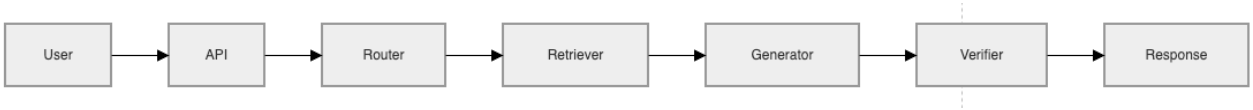



Рис.3. Пайплайн подсистемы мониторинга и трейсинга

Трейсинг осуществляется на следующих уровнях:

- API-слой
  - Регистрация запросов

    Фиксирует факт обращения → база для аудита, метрик нагрузки и расследований.
  - trace_id

Уникальный ID запроса → связывает все шаги в одну цепочку (основа distributed tracing).

- Время начала

  Точка отсчёта latency → позволяет считать SLA и находить деградации.

- Оркестрация агентов
  - Последовательность вызовов

    Показывает, какие агенты и в каком порядке отработали → выявляет ошибки логики pipeline.
  - Вход/выход каждого шага

    Даёт полную прослеживаемость данных → можно понять, где “сломался” ответ.
  - Контроль маршрутизации

    Проверяет решения Router → позволяет ловить ошибки intent-классификации.

- Внешние зависимости
  - LLM-вызовы

    Контроль prompt, ответа, токенов и latency → управление качеством и стоимостью.
  - Vector DB

    Логирует найденные документы и их score → позволяет диагностировать retrieval.
  - Tools

    Фиксирует вызовы внешних действий → контроль side-effects и интеграций.

Такое покрытие позволяет восстановить полный путь обработки запроса и определить источник возможной ошибки или деградации качества.

### 9.5. Конфигурация подсистемы

Настройка трейсинга осуществляется через переменные окружения:

- LANGFUSE_SECRET_KEY – секретный ключ доступа;
- LANGFUSE_PUBLIC_KEY – публичный ключ;
- LANGFUSE_BASE_URL – адрес сервера трейсинга.

Использование переменных окружения обеспечивает:

- независимость конфигурации от кода;
- удобство деплоя в различных средах;
- возможность безопасного управления доступом.

### 9.6. Сценарий эксплуатации

Процесс использования подсистемы включает следующие этапы:

1. Развертывание экземпляра Langfuse (локально или в облаке);
2. Настройка переменных окружения;
3. Запуск приложения;
4. Анализ трейсингов через веб-интерфейс.

В процессе эксплуатации доступны:

- подробные трейсы запросов;
- временные диаграммы выполнения;
- статистика по ошибкам и задержкам.

### 9.7. Роль в обеспечении надежности системы

Подсистема мониторинга и трейсинга играет ключевую роль в обеспечении надежности разрабатываемого решения.

Основные эффекты внедрения:

- Повышение прозрачности - возможность анализа каждого этапа обработки запроса.
- Улучшение качества - выявление ошибок и некорректных ответов.
- Снижение рисков - быстрое обнаружение аномалий в работе системы.
- Поддержка верификации - использование трейсинговых данных для анализа достоверности ответов.

### 9.8. Архитектурные преимущества

Интеграция observability-подсистемы соответствует современным принципам проектирования распределённых систем:

- Observability by design – наблюдаемость заложена на этапе проектирования;
- Separation of concerns – мониторинг вынесен в отдельный слой;
- Trace-driven debugging – отладка на основе трасс выполнения;
- Data-driven improvement – развитие системы на основе накопленных данных.

### 9.9. Ограничения и направления развития

Несмотря на преимущества, подсистема имеет ряд ограничений:

- дополнительная нагрузка на систему;
- необходимость хранения трейсинговых данных;
- зависимость от внешнего сервиса.

Перспективы развития включают:

- автоматическую оценку качества ответов (LLM-as-a-judge);
- интеграцию с системой алертинга;
- построение SLA/SLI метрик;
- использование трейсингов для обучения и оптимизации агентов.

Подсистема мониторинга и трейсинга на базе Langfuse является критически важным элементом архитектуры разрабатываемого ассистента. Она обеспечивает наблюдаемость, трассируемость и контроль качества работы системы, что особенно важно для применения в условиях атомной энергетики и других критически значимых областях.

# 10. ЭКСПЕРИМЕНТАЛЬНАЯ ЧАСТЬ

## 10.1. Методика тестирования и валидации системы

### 10.1.1. Цели тестирования

В условиях применения LLM и RAG-архитектуры в критически значимых системах (таких как поддержка операторов АЭС) тестирование должно обеспечивать не только функциональную корректность, но и контроль качества генерации.

Основные цели тестирования:

- верификация корректности работы pipeline системы;
- оценка качества retrieval и генерации;
- выявление деградаций при изменении компонентов;
- обеспечение воспроизводимости результатов;
- формирование базы для последующего SLA/SLI мониторинга.

Тестирование рассматривается как часть архитектуры системы, а не как вспомогательный этап разработки.

### 10.1.2. Архитектура тестирования

Подсистема тестирования реализована как отдельный слой, интегрированный в процесс разработки и эксплуатации системы.

Ключевые принципы:

- Separation of concerns – тестирование отделено от бизнес-логики;
- Deterministic + probabilistic validation – сочетание строгих и вероятностных проверок;
- LLM-aware testing – учет недетерминированности генеративных моделей;
- Automation-first – полная автоматизация запуска и оценки тестов.

Архитектурно тестирование покрывает весь pipeline:

### 10.1.3. Используемые инструменты

Для реализации тестирования используются следующие инструменты:

- pytest – основной фреймворк автотестирования;
- DeepEval – библиотека для оценки качества ответов LLM;

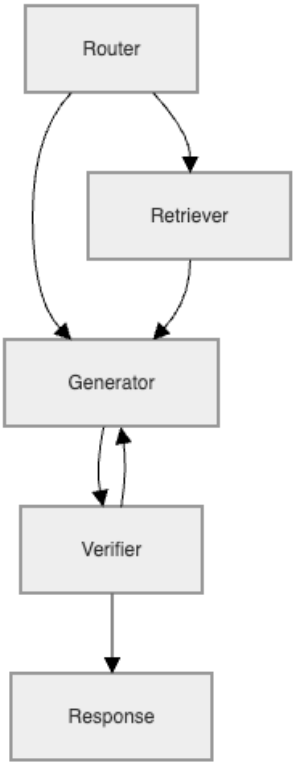



Рис.4. Пайплайн тестирования

- Ollama – локальное выполнение моделей для LLM-as-a-Judge.

Данный стек обеспечивает:

- декларативное описание тестов;
- масштабируемость тестового набора;
- возможность интеграции в CI/CD;
- воспроизводимость результатов в изолированной среде.

### 10.1.4. Режимы тестирования

Система поддерживает два режима тестирования, оптимизированных под разные задачи.

Быстрые тесты (fast mode)

Назначение:

- быстрая проверка корректности системы при разработке;
- контроль регрессий.

Характеристики:

- не используют LLM-as-a-Judge;
- выполняются за ~1 минуту;
- основаны на детерминированных проверках.

Проверяемые аспекты:

- корректность маршрутизации (Router);
- стабильность retrieval;
- базовая структура ответа;
- отсутствие критических ошибок.

Запуск:

*pytest -m "not slow»*

где:

pytest – инструмент автоматического тестирования для Python, представляющий собой CLI-приложение и библиотеку, обеспечивающие обнаружение, запуск и оценку тестов, а также формирование отчётов о результатах их выполнения;

-m “not slow” – параметр фильтрации тестов по маркерам, при котором выполняются только те тесты, которые не помечены маркером slow (как правило, длительные или ресурсоёмкие сценарии), что позволяет ускорить процесс проверки системы при разработке.

Полные тесты (full mode)

Назначение:

- комплексная оценка качества системы;
- проверка релевантности и семантики ответов.

Характеристики:

- используют LLM-as-a-Judge;
- выполняются за 5–10 минут;
- требуют доступной модели (через Ollama).

Проверяемые аспекты:

- соответствие ответа запросу;
- полнота и релевантность;
- корректность использования контекста;
- отсутствие hallucinations.

### 10.1.5. Метрики качества

Для оценки качества используются метрики уровня LLM.

Ключевая метрика:

- Answer Relevancy Metric – оценивает, насколько ответ соответствует запросу пользователя.

Подход:

- эталонный ответ или критерии задаются в тесте;
- LLM выступает в роли judge-модели (модели-судьи);
- результат выражается в числовой оценке (score).

Дополнительно могут использоваться:

- faithfulness (соответствие источникам);
- contextual precision (точность использования контекста);
- consistency (согласованность ответа).

### 10.1.6. Структура тестового набора

Тестовый набор формируется как коллекция сценариев, отражающих реальные задачи оператора.

Типы сценариев:

- поиск информации;
- интерпретация инструкций;
- анализ ситуаций;
- комбинированные запросы (retrieval + reasoning).

Каждый тест включает:

- входной запрос;
- ожидаемые свойства ответа;
- критерии оценки;
- (опционально) эталонный ответ.

### 10.1.7. Процесс выполнения тестов

Процесс выполнения тестов представляет собой формализованный и воспроизводимый pipeline, обеспечивающий контроль качества системы на всех этапах её развития. Он интегрирован в общий жизненный цикл разработки и ориентирован на выявление как функциональных, так и качественных отклонений в работе LLM-компонентов.

Процесс включает следующие этапы:

1. Подготовка окружения

На данном этапе обеспечивается воспроизводимость тестовой среды и корректность зависимостей:

- выполняется установка и синхронизация зависимостей проекта (через uv/lock-файл);
- инициализируется тестовое окружение (переменные среды, доступ к БД, API);
- настраивается модель, используемая в DeepEval (в том числе параметры inference и endpoint).

Данный этап критически важен для устранения вариативности результатов и обеспечения сопоставимости прогонов.

2. Запуск тестов

Тестирование выполняется в одном из двух режимов:

- быстрый режим (fast mode) – для оперативной проверки корректности и регрессий;
- полный режим (full mode) – для комплексной оценки качества с использованием LLM-as-a-Judge.

Запуск осуществляется через единый entrypoint (pytest), что обеспечивает унификацию и удобство интеграции в CI/CD.

3. Сбор результатов

Результаты тестирования агрегируются в структурированном виде:

- для детерминированных тестов фиксируется статус pass/fail;
- для LLM-тестов рассчитываются метрики качества (relevancy, faithfulness и др.);
- при необходимости сохраняются промежуточные данные (логи, ответы модели, контекст).

Это формирует базу для последующего анализа и построения метрик качества системы.

4. Анализ результатов

На этапе анализа выполняется интерпретация полученных данных:

- выявляются деградации качества при изменениях системы (regression detection);
- проводится локализация проблемных компонентов (Router, Retriever, Generator, Verifier);
- сопоставляются результаты с предыдущими прогонами и целевыми метриками.

В связке с системой трейсинга (Langfuse) данный этап позволяет перейти от простой фиксации ошибок к их причинно-следственному анализу.

### 10.1.8. Интеграция с архитектурой системы

Подсистема тестирования интегрирована с:

- pipeline обработки запросов – тестируется end-to-end;
- подсистемой трейсинга (Langfuse) – используется для анализа поведения;
- подсистемой верификации – дополняет runtime-проверки offline-оценкой.

Это обеспечивает:

- замкнутый цикл контроля качества;
- сопоставимость offline и online метрик;
- возможность trace-driven анализа ошибок.

### 10.1.9. Ограничения подхода

Несмотря на архитектурные преимущества и высокий уровень автоматизации, предложенная методика тестирования имеет ряд ограничений, обусловленных спецификой LLM-систем и используемых инструментов оценки.

Ключевым ограничением является зависимость от качества judge-модели, используемой в рамках подхода LLM-as-a-Judge. Оценка результатов в данном случае носит вероятностный характер и напрямую определяется способностью модели корректно интерпретировать критерии качества. Ошибки или смещения judge-модели могут приводить к некорректной оценке даже при формально правильных ответах системы.

Дополнительно, LLM-оценки характеризуются потенциальной нестабильностью. При одинаковых входных данных возможны вариации результатов, что усложняет интерпретацию метрик и требует использования усреднения, повторных прогонов или калибровки оценок.

Использование LLM в контуре тестирования также приводит к увеличению времени выполнения тестов. Полный режим (full mode) требует значительных вычислительных ресурсов и увеличивает latency, что ограничивает частоту запуска и усложняет интеграцию в быстрые CI-пайплайны.

Отдельным ограничением является необходимость поддержки тестового корпуса (совокупность тестовых сценариев, включающих входные запросы, ожидаемые свойства ответов и критерии оценки, используемая для проверки корректности и качества работы системы). Тесты должны отражать реальные сценарии использования системы, что требует постоянного обновления, расширения и валидации набора тест-кейсов по мере эволюции системы и предметной области.

В совокупности данные ограничения требуют балансировки между точностью оценки, стоимостью вычислений и скоростью обратной связи.

### 10.1.10. Перспективы развития

Дальнейшее развитие подсистемы тестирования направлено на переход от изолированной проверки к интегрированной системе управления качеством LLM-компонентов на уровне всей архитектуры.

Одним из ключевых направлений является внедрение CI/CD-пайплайнов с автоматическим прогоном тестов. Это позволит обеспечить непрерывный контроль качества при каждом изменении системы и сократить время обнаружения регрессий.

Развитие подхода включает построение regression benchmarking - процесс систематического сравнения результатов тестирования системы с предыдущими версиями с целью выявления деградаций качества и оценки влияния внесённых изменений. Это обеспечивает количественную оценку деградаций и позволяет принимать решения на основе исторических данных.

Введение SLA/SLI- (см. 1.22, 1.23, 1,24) качества ответов переводит тестирование в плоскость эксплуатационных показателей. Формализация целевых значений (например, минимальный уровень relevancy или максимальный latency) позволяет интегрировать LLM-компоненты в корпоративные стандарты качества.

Автоматическая генерация тестов (synthetic evals) рассматривается как способ масштабирования тестового покрытия. Генерация сценариев на основе LLM позволяет быстро расширять корпус тестов, снижая зависимость от ручного труда.

Использование production-трейсов (например, из Langfuse) открывает возможность формирования тест-кейсов на основе реальных пользовательских запросов. Это повышает релевантность тестирования и обеспечивает замкнутый цикл улучшения системы.

Финальным этапом эволюции является переход к модели continuous evaluation(процесс непрерывной оценки качества ответов LLM-системы на основе тестов и production-трафика, направленный на выявление деградаций и контроль метрик качества), объединяющей offline и online оценку. В этом подходе тестирование становится непрерывным процессом, встроенным в эксплуатацию системы, с постоянным сбором метрик, анализом качества и автоматической реакцией на деградации.

Таким образом, подсистема тестирования трансформируется из инструмента проверки в полноценный механизм управления качеством и надежностью LLM-систем на уровне платформы.

#### 10.1.11. Роль в архитектуре

Подсистема тестирования является ключевым элементом обеспечения качества и надежности системы.

Она обеспечивает:

- контролируемое развитие системы;
- снижение риска деградации;
- формализацию качества LLM-компонентов;
- основу для промышленной эксплуатации.

В долгосрочной перспективе тестирование трансформируется в систему управления качеством LLM (LLM Quality Management System), интегрированную с observability, CI/CD и эксплуатационными метриками.

## 11. ОЦЕНКА РЕЗУЛЬТАТОВ

В ходе разработки и экспериментальной валидации системы достигнуты следующие результаты.

Обеспечена стабильность retrieval-подсистемы. Поиск релевантных документов демонстрирует предсказуемое поведение и достаточную точность, что подтверждается метриками (precision@k, recall@k). Использование векторного поиска в сочетании с fallback-механизмами позволяет минимизировать случаи отсутствия релевантного контекста.

Достигнута корректность генерации ответов. Языковая модель формирует ответы, соответствующие пользовательскому запросу и использующие предоставленный контекст. Применение prompt engineering и ограничений на структуру ответа позволяет снизить количество некорректных или нерелевантных генераций.

Реализована базовая подсистема верификации. Внедрены проверки источников, числовых значений и логической согласованности, что позволяет выявлять часть ошибок на этапе post-processing. Это формирует первый уровень защиты от некорректных ответов и повышает достоверность и согласованность выходных данных системы.

В совокупности полученные результаты подтверждают работоспособность выбранной архитектуры и её применимость для задач поиска и генерации в доменно-ограниченной среде.

## 12. ОГРАНИЧЕНИЯ

Несмотря на достигнутые результаты, система имеет ряд ограничений, обусловленных как архитектурными решениями, так и природой используемых моделей.

Система демонстрирует зависимость от тестового и обучающего корпуса. Качество retrieval и генерации напрямую определяется полнотой, актуальностью и структурой базы знаний. Недостатки корпуса (неполнота, шум, устаревшие данные) приводят к снижению качества ответов.

Сохраняется влияние ошибок языковой модели (LLM). Даже при наличии релевантного контекста возможны hallucinations, логические неточности или некорректная интерпретация информации. Это связано с вероятностной природой генерации и не может быть полностью устранено на текущем уровне развития технологий.

Дополнительным ограничением является latency системы. Многоступенчатый pipeline (retrieval → generation → verification) увеличивает время ответа, особенно при использовании LLM в контуре оценки или верификации. Это накладывает ограничения на применение системы в сценариях с жесткими требованиями к времени отклика.

Данные ограничения определяют направления дальнейшего развития системы, включая улучшение качества корпуса, усиление верификации и оптимизацию производительности.

## 13. ЗАКЛЮЧЕНИЕ

В рамках данной работы разработан прототип интеллектуальной системы информационной поддержки оператора, обеспечивающая доступ к знаниям и источникам для операторов атомных электростанций, реализованный на основе архитектурного подхода Retrieval-Augmented Generation (RAG) и агентной модели обработки запросов.

В ходе выполнения работы были решены следующие задачи:

- проведён анализ предметной области эксплуатации АЭС и выявлены ключевые требования к системам поддержки операторов;
- обоснован выбор архитектуры, обеспечивающей достоверность, трассируемость и модульность системы;
- разработана архитектура программного решения, включающая компоненты маршрутизации, поиска, генерации и верификации;
- реализован прототип системы с использованием современных инструментов обработки естественного языка и векторного поиска;
- внедрена подсистема верификации, обеспечивающая базовый контроль корректности формируемых ответов;
- реализована подсистема мониторинга и трейсинга, позволяющая анализировать поведение системы и обеспечивать её наблюдаемость;
- проведена экспериментальная оценка качества работы системы, подтвердившая её работоспособность и применимость для решения задач поиска и генерации в доменно-ограниченной (см. 1.27) среде.

Полученные результаты демонстрируют, что применение RAG-подхода в сочетании с агентной архитектурой позволяет обеспечить приемлемый уровень релевантности и интерпретируемости (см.1.28) ответов, а также создать основу для построения систем информационной поддержки оператора, обеспечивающих доступ к знаниям и источникам в условиях повышенных требований к надежности.

Вместе с тем разработанный прототип имеет ряд ограничений, связанных с зависимостью от качества базы знаний, вероятностной природой генеративных моделей и дополнительными временными затратами на обработку запросов. Указанные ограничения определяют направления дальнейшего развития системы.

К основным перспективам развития разработанного решения относятся:

- интеграция в контур автоматизированных систем управления технологическими процессами (АСУ ТП), что позволит использовать систему в качестве вспомогательного инструмента анализа и интерпретации данных в реальных условиях эксплуатации. Данное направление требует разработки механизмов безопасной интеграции с промышленными интерфейсами, обеспечения изоляции и соблюдения требований к надежности и отказоустойчивости;
- расширение подсистемы верификации, предполагающее переход от базовых проверок к многоуровневой системе контроля достоверности, включающей формализованные правила, семантическую проверку, а также механизмы согласования результатов между компонентами системы. Развитие данного направления направлено на снижение вероятности ошибок и повышение доверия к результатам работы системы;
- реализация концепции human-in-the-loop (см.1.29), предусматривающей включение оператора в контур принятия решений. Это позволит обеспечить дополнительный уровень контроля, повысить качество ответов за счёт экспертной обратной связи и сформировать механизм накопления и актуализации знаний системы на основе практического опыта эксплуатации.

Таким образом, разработанный прототип представляет собой основу для дальнейшего развития в направлении промышленной системы поддержки операторов, соответствующей требованиям безопасности, трассируемости и управляемости, предъявляемым к программным решениям в области атомной энергетики.

## 14. СПИСОК ЛИТЕРАТУРЫ

1. Wooldridge M. An Introduction to MultiAgent Systems. – Wiley, 2009.

2. Fielding R. Architectural Styles and the Design of Network-based Software Architectures (REST). – 2000.
3. Humble J., Farley D. Continuous Delivery. – Addison-Wesley, 2010.
4. Merkel D. Docker: Lightweight Linux Containers for Consistent Development and Deployment. – Linux Journal, 2014.
5. Elmasri R., Navathe S. Fundamentals of Database Systems. – Pearson, 2016.
6. Mikolov T. et al. Efficient Estimation of Word Representations in Vector Space. – 2013.
7. Langfuse Documentation – https://langfuse.com
8. Brown T. et al. Language Models are Few-Shot Learners. – NeurIPS, 2020.
9. Zheng L. et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. – 2023.
10. Python Packaging Authority. Python Packaging User Guide.
11. Sigelman B. et al. Dapper, a Large-Scale Distributed Systems Tracing Infrastructure. – Google, 2010.
12. Dean J., Ghemawat S. MapReduce: Simplified Data Processing on Large Clusters. – 2004.
13. PostgreSQL Global Development Group – https://www.postgresql.org
14. pgvector Documentation – https://github.com/pgvector/pgvector
15. Lewis P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. – 2020.